

```
In [1]: #SD7502 Assignment 2
```

```
In [2]: #1: Loading and analysing data
```

```
In [3]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

#Loading the datasets
training_set_path = "F:\\Jordan\\Documents\\Github\\SD7502-Project2-2220115_2220145\\D
testing_set_path = "F:\\Jordan\\Documents\\Github\\SD7502-Project2-2220115_2220145\\D

#Loading the CSV files into dataframes
train_df = pd.read_csv(training_set_path)
test_df = pd.read_csv(testing_set_path)

#Displaying basic info regarding both datasets
print("Info for Training set")
train_df.info()

print("\nInfo for Testing set")
test_df.info()

#Displaying first 5 rows for each set
print("\nFirst 5 rows of Training set:")
print(train_df.head())

print("\nFirst 5 rows of Testing set:")
print(test_df.head())

#Checking for missing values
print("\nMissing values in Training set:")
print(train_df.isnull().sum())

print("\nMissing values in Testing set:")
print(test_df.isnull().sum())

#Class distribution in the training set - Important to indicate how many examples belong to each class
class_distribution = train_df['label'].value_counts()

#Plotting the summary
plt.figure(figsize=(14, 10))
sns.barplot(y=class_distribution.index, x=class_distribution.values, palette="viridis")
plt.xlabel("Number of Images")
plt.ylabel("Species")
plt.xticks(rotation=0)
plt.yticks(fontsize=10)
plt.tight_layout()
plt.show()
```

```

Info for Training set
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6499 entries, 0 to 6498
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   filename    6499 non-null   object
1   label       6499 non-null   object
dtypes: object(2)
memory usage: 101.7+ KB

```

```

Info for Testing set
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2786 entries, 0 to 2785
Data columns (total 1 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   filename    2786 non-null   object
dtypes: object(1)
memory usage: 21.9+ KB

```

```

First 5 rows of Training set:
      filename      label
0  Image_1.jpg  SOUTHERN DOGFACE
1  Image_2.jpg      ADONIS
2  Image_3.jpg  BROWN SIPROETA
3  Image_4.jpg      MONARCH
4  Image_5.jpg  GREEN CELLED CATTLEHEART

```

```

First 5 rows of Testing set:
      filename
0  Image_1.jpg
1  Image_2.jpg
2  Image_3.jpg
3  Image_4.jpg
4  Image_5.jpg

```

```

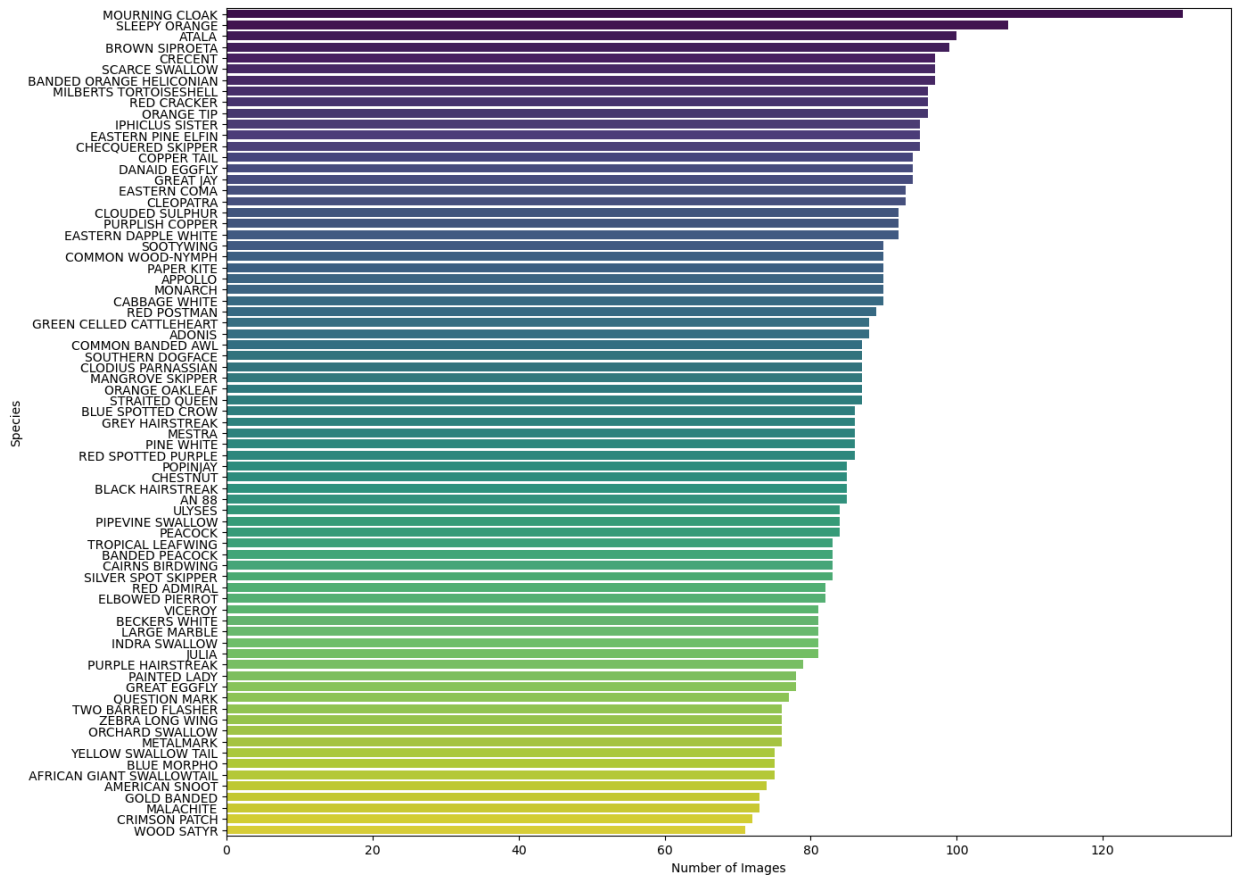
Missing values in Training set:
filename    0
label       0
dtype: int64

```

```

Missing values in Testing set:
filename    0
dtype: int64

```



In [4]: #2A Define a Data Loader class and transformation pipeline

```
In [5]: import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
import pandas as pd

# Defining the transformation pipeline for preprocessing images
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# Modified ButterflyDataset class to handle cases with or without labels
class ButterflyDataset(Dataset):
    def __init__(self, csv_file, root_dir, transform=None, has_labels=True):
        self.data = pd.read_csv(csv_file)
        self.root_dir = root_dir
        self.transform = transform
        self.has_labels = has_labels

        if has_labels:
            # Encode the labels into integers
            self.data['label'], self.label_mapping = pd.factorize(self.data['label'])

    def __len__(self):
        return len(self.data)
```

```

def __getitem__(self, idx):
    img_name = f"{self.root_dir}/{self.data.iloc[idx, 0]}"
    image = Image.open(img_name).convert("RGB")

    if self.has_labels:
        # Get the Label if available
        label = self.data.iloc[idx, 1] # This will now be an integer
        if self.transform:
            image = self.transform(image)
        return image, label
    else:
        # Return only the image if no Labels are available
        if self.transform:
            image = self.transform(image)
        return image

# Initialize Dataset and DataLoader for the training set (has Labels)
train_dataset = ButterflyDataset(
    csv_file='F:\\Jordan\\Documents\\Github\\SD7502-Project2-2220115_2220145\\Datasets',
    root_dir='F:\\Jordan\\Documents\\Github\\SD7502-Project2-2220115_2220145\\Datasets',
    transform=transform,
    has_labels=True # Indicates that this dataset has Labels
)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

# Initialize Dataset and DataLoader for the testing set (no Labels)
test_dataset = ButterflyDataset(
    csv_file='F:\\Jordan\\Documents\\Github\\SD7502-Project2-2220115_2220145\\Datasets',
    root_dir='F:\\Jordan\\Documents\\Github\\SD7502-Project2-2220115_2220145\\Datasets',
    transform=transform,
    has_labels=False # Indicates that this dataset does not have Labels
)

test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

print("Data loaders initialized successfully!")

```

Data loaders initialized successfully!

In [6]: #2B Training, Validating, and Testing the models

```

In [7]: from torch.utils.data import random_split

#Loading the full training set
full_train_dataset = ButterflyDataset(csv_file='F:\\Jordan\\Documents\\Github\\SD7502-Project2-2220115_2220145\\Datasets',
                                       root_dir='F:\\Jordan\\Documents\\Github\\SD7502-Project2-2220115_2220145\\Datasets',
                                       transform=transform)

#Defining the split ratios
train_size = int(0.8 * len(full_train_dataset)) #80% for training
val_size = len(full_train_dataset) - train_size #20% for validation (remainder)

#Splitting the dataset into training and validation sets
train_dataset, val_dataset = random_split(full_train_dataset, [train_size, val_size])

#Preparing DataLoaders for training, validation, and test sets
#NOTE: Training and Test sets have been prepared in the previous cell
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

```

```
#Printing the datasets
print(f"Training set size: {len(train_dataset)}")
print(f"Validation set size: {len(val_dataset)}")
print(f"Test set size: {len(test_dataset)}")
```

```
Training set size: 5199
Validation set size: 1300
Test set size: 2786
```

In [8]: *#2C Developing deep Learning models to construct classification models*

In [9]: *#Convolutional Neural Network (CNN)*

```
#CNNs is a deep-learning model which performs best in tasks involving image data. CNNs
#These layers apply filters to detect patterns such as textures, edges, and complicated
#convolutional layers which are then followed by an activation function, e.g., ReLU (ReLU)
#dimensionality. Overall, CNN's are highly efficient for image processing which makes
```

In [10]: **import** torch.nn **as** nn

```
class CNNModel(nn.Module):
    def __init__(self, num_classes=75):
        super(CNNModel, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.fc1 = nn.Linear(64 * 16 * 16, 128) # Adjust this based on the input size
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # After conv1 -> pool
        x = self.pool(F.relu(self.conv2(x))) # After conv2 -> pool
        x = self.pool(F.relu(self.conv3(x))) # After conv3 -> pool

        # Dynamically calculate the flattening size
        x = x.view(x.size(0), -1) # Flatten the tensor
        x = F.relu(self.fc1(x)) # Pass through the fully connected layer
        x = self.fc2(x) # Final output layer
        return x

print("Complete!")
```

Complete!

In [11]: *#Multilayer Perceptron (MLP)*

```
#An MLP is a fully connected feedforward neural network. It comprises of an input layer
#MLP's treat the input data as a one-dimensional vector, meaning the spatial information is lost
#Every neuron in MLP is connected to each neuron in the following layer. Activations are passed
#Although MLPs can be used to handle image data, they tend to lack the efficiency and accuracy of CNNs
#This is due to the fact that they are unable to exploit the spatial structure of the input data
#Despite this, however, MLPs are still a popular choice for comparing things across a wide range of
```

In [12]: **class** MLPModel(nn.Module):
 def __init__(self, num_classes=75): # Update the number of classes to 75
 super(MLPModel, self).__init__()

```

self.fc1 = nn.Linear(3 * 128 * 128, 512) # Adjust input size based on image c
self.fc2 = nn.Linear(512, 128)
self.fc3 = nn.Linear(128, num_classes)

def forward(self, x):
    # Flattening the input dynamically
    x = x.view(x.size(0), -1) # Dynamically flatten the tensor
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    x = self.fc3(x)
    return x

print("complete!")

```

complete!

In [13]: *#Recurrent Neural Network (RNN) with LSTM*

*#An RNN with LSTM units are a neural network architecture which is typically used for
#time series, or for audio. LSTM networks are designed in a way where they can capture
#a memory of past inputs through their cells. When using this in the context of image
#rows of an images as sequences to then be fed in the LSTM to model sequential or spat*

*#Although this approach is not typically used for images, it allows the network to pro
#to more interesting insights when compared to results from CNN models for example. AL
#to the other models, it's more useful when exploring as to how sequence models perfor*

In [14]: **class** RNNModel(nn.Module):

```

    def __init__(self, num_classes=75, input_size=64, hidden_size=128, num_layers=2):
        super(RNNModel, self).__init__()
        self.lstm = nn.LSTM(input_size=input_size, hidden_size=hidden_size, num_layers=
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # Dynamically adjust input to LSTM (batch_size, sequence_length, input_size)
        batch_size = x.size(0)
        x = x.view(batch_size, -1, 64) # Reshape to (batch_size, sequence_length, inp

        lstm_out, _ = self.lstm(x)
        lstm_out = lstm_out[:, -1, :] # Get the output of the last time step

        x = self.fc(lstm_out) # Pass through the fully connected layer
        return x

print("complete")

```

complete

In [15]: *#2D Training the models using Epochs*

In [16]: *# Check the output of one batch from the train_loader*

```

batch = next(iter(train_loader))
print(type(batch))
print(batch)

# Adjusted training function to unpack only inputs and labels
def train_model(model, train_loader, val_loader, criterion, optimizer, num_epochs=10):
    train_losses = []
    val_losses = []

```

```

best_val_loss = float("inf")
patience = 5
patience_counter = 0

for epoch in range(num_epochs):
    # Training phase
    model.train()
    running_loss = 0.0
    for batch in train_loader:
        # Assuming batch is a tuple of (inputs, labels)
        inputs, labels = batch[0], batch[1]

        # Ensure labels are numerical (e.g., integers representing class indices)
        if isinstance(labels[0], str):
            raise ValueError("Labels should be numerical indices, but got strings.")

        # Move data to the correct device (GPU or CPU)
        inputs, labels = inputs.to(device), labels.to(device)

        # Zero the parameter gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        # Backward pass and optimization
        loss.backward()
        optimizer.step()

    running_loss += loss.item() * inputs.size(0)

    epoch_train_loss = running_loss / len(train_loader.dataset)
    train_losses.append(epoch_train_loss)

    # Validation phase
    model.eval()
    running_val_loss = 0.0
    with torch.no_grad():
        for batch in val_loader:
            inputs, labels = batch[0], batch[1]

            # Ensure labels are numerical
            if isinstance(labels[0], str):
                raise ValueError("Labels should be numerical indices, but got strings")

            inputs, labels = inputs.to(device), labels.to(device)

            outputs = model(inputs)
            loss = criterion(outputs, labels)

            running_val_loss += loss.item() * inputs.size(0)

    epoch_val_loss = running_val_loss / len(val_loader.dataset)
    val_losses.append(epoch_val_loss)

    print(f"Epoch {epoch + 1}/{num_epochs}, Train Loss: {epoch_train_loss:.4f}, Val Loss: {epoch_val_loss:.4f}")

    # Early stopping
    if epoch_val_loss < best_val_loss:

```

```
        best_val_loss = epoch_val_loss
        patience_counter = 0
    else:
        patience_counter += 1

    if patience_counter >= patience:
        print("Early stopping triggered")
        break

    return train_losses, val_losses
```



```

<class 'list'>
[tensor([[[[-0.6109, -0.6281, -0.6109, ..., 0.3994, 0.3994, 0.4166],
  [-0.6281, -0.6281, -0.6109, ..., 0.3994, 0.3994, 0.4166],
  [-0.6281, -0.6281, -0.5938, ..., 0.3994, 0.3994, 0.4166],
  ...,
  [ 1.8893, 1.8722, 1.8893, ..., 1.8722, 1.9235, 1.9578],
  [ 1.8893, 1.8893, 1.9064, ..., 1.8722, 1.8550, 1.8722],
  [ 1.9064, 1.9064, 1.9064, ..., 1.9749, 1.9578, 1.9235]]],

  [[[-0.2850, -0.3025, -0.2850, ..., 0.8529, 0.8529, 0.8704],
  [-0.3025, -0.3025, -0.2850, ..., 0.8529, 0.8529, 0.8704],
  [-0.3025, -0.3025, -0.2675, ..., 0.8529, 0.8529, 0.8704],
  ...,
  [ 2.0434, 2.0259, 2.0084, ..., 1.9909, 2.0434, 2.0609],
  [ 2.0434, 2.0434, 2.0609, ..., 1.9909, 1.9909, 2.0084],
  [ 2.0609, 2.0609, 2.0609, ..., 2.0959, 2.0959, 2.0609]]],

  [[[-1.2119, -1.2293, -1.2119, ..., 0.8274, 0.8448, 0.8797],
  [-1.2293, -1.2293, -1.2119, ..., 0.8274, 0.8448, 0.8797],
  [-1.2293, -1.2293, -1.1944, ..., 0.8274, 0.8622, 0.8797],
  ...,
  [ 1.9254, 1.9080, 1.9080, ..., 2.0125, 2.0300, 2.0125],
  [ 1.9428, 1.9428, 1.9603, ..., 2.0474, 1.9777, 1.9254],
  [ 1.9603, 1.9603, 1.9603, ..., 2.1520, 2.0823, 1.9603]]],

  [[[ 2.1633, 2.1633, 2.1290, ..., 2.1804, 2.2147, 2.2318],
  [ 2.1290, 2.1290, 2.1290, ..., 2.1633, 2.2147, 2.2318],
  [ 2.0948, 2.0948, 2.0948, ..., 2.1633, 2.1975, 2.2147],
  ...,
  [ 2.1462, 2.1633, 2.1633, ..., 2.1633, 2.1975, 2.1975],
  [ 2.1290, 2.1633, 2.1633, ..., 2.1633, 2.1975, 2.2147],
  [ 2.1804, 2.1633, 2.1804, ..., 2.1633, 2.1804, 2.1975]]],

  [[[ 2.3410, 2.3410, 2.3060, ..., 2.3585, 2.3936, 2.4111],
  [ 2.3060, 2.3060, 2.3060, ..., 2.3410, 2.3936, 2.4111],
  [ 2.2710, 2.2710, 2.2710, ..., 2.3410, 2.3761, 2.3936],
  ...,
  [ 2.3235, 2.3410, 2.3410, ..., 2.3410, 2.3761, 2.3761],
  [ 2.3060, 2.3410, 2.3410, ..., 2.3410, 2.3761, 2.3936],
  [ 2.3585, 2.3410, 2.3585, ..., 2.3410, 2.3585, 2.3761]]],

  [[[ 2.5529, 2.5529, 2.5180, ..., 2.5703, 2.6051, 2.6226],
  [ 2.5180, 2.5180, 2.5180, ..., 2.5529, 2.6051, 2.6226],
  [ 2.4831, 2.4831, 2.4831, ..., 2.5529, 2.5877, 2.6051],
  ...,
  [ 2.5529, 2.5529, 2.5529, ..., 2.5529, 2.5877, 2.5877],
  [ 2.5180, 2.5529, 2.5529, ..., 2.5529, 2.5877, 2.6051],
  [ 2.5703, 2.5529, 2.5703, ..., 2.5529, 2.5703, 2.5877]]],

  [[[-1.2445, -1.1247, -0.9705, ..., -1.5185, -1.6213, -1.7240],
  [-1.2788, -1.1760, -1.0390, ..., -1.4672, -1.6727, -1.7412],
  [-1.3302, -1.2445, -1.1247, ..., -1.6213, -1.7069, -1.7412],
  ...,
  [-0.2856, -0.3369, -0.0629, ..., -0.1999, 0.3994, 0.9132],
  [ 0.0741, -0.0287, 0.1768, ..., -0.9192, -0.5253, -0.3369],
  [ 0.5022, 0.3823, 0.5707, ..., -1.2959, -0.6281, -0.3712]],

  [[[-1.0028, -0.9153, -0.7577, ..., -1.1253, -1.2654, -1.3880],

```

```

[-1.0728, -0.9678, -0.7752, ..., -1.1078, -1.3354, -1.4055],
[-1.1779, -1.0553, -0.8627, ..., -1.2829, -1.4055, -1.4405],
...,
[-0.5651, -0.6176, -0.2675, ..., -0.5301, 0.0826, 0.6078],
[-0.1975, -0.4251, -0.2500, ..., -1.0028, -0.6352, -0.4951],
[ 0.0826, -0.1800, -0.0749, ..., -1.2129, -0.7227, -0.6001]],

[[-1.3687, -1.2816, -1.1596, ..., -1.5779, -1.6824, -1.7870],
 [-1.3861, -1.2990, -1.1770, ..., -1.5430, -1.7522, -1.7870],
 [-1.4210, -1.3687, -1.2467, ..., -1.6999, -1.7696, -1.7870],
 ...,
 [-1.3339, -1.4907, -1.3861, ..., -1.5779, -1.4384, -1.1247],
 [-1.3687, -1.4907, -1.3687, ..., -1.6824, -1.6824, -1.7347],
 [-1.2816, -1.3861, -1.1944, ..., -1.7173, -1.6999, -1.7870]]],

...,

[[[-0.0801, -0.0458, -0.0287, ..., -0.0287, -0.0287, -0.0287],
 [-0.0458, -0.0287, -0.0458, ..., -0.0287, -0.0287, -0.0287],
 [-0.0629, -0.0458, -0.0458, ..., 0.0056, -0.0116, -0.0287],
 ...,
 [ 0.0912, 0.0912, 0.1083, ..., 1.7352, 1.4440, 1.3755],
 [ 0.0912, 0.0741, 0.0741, ..., 1.7523, 1.4440, 1.4783],
 [ 0.0741, 0.0569, 0.0569, ..., 1.5125, 1.6495, 1.7523]],

[[ 0.9055, 0.9405, 0.9580, ..., 0.9230, 0.9230, 0.9230],
 [ 0.9405, 0.9580, 0.9405, ..., 0.9230, 0.9230, 0.9230],
 [ 0.9230, 0.9405, 0.9405, ..., 0.9580, 0.9405, 0.9230],
 ...,
 [ 1.0805, 1.0805, 1.0980, ..., 1.6583, 1.3782, 1.3081],
 [ 1.0805, 1.0805, 1.1155, ..., 1.6232, 1.3081, 1.3256],
 [ 1.0805, 1.0805, 1.0980, ..., 1.4132, 1.5357, 1.6408]],

[[ 1.8731, 1.9080, 1.9254, ..., 1.9080, 1.9080, 1.9080],
 [ 1.9080, 1.9254, 1.9080, ..., 1.9080, 1.9080, 1.9080],
 [ 1.8905, 1.9080, 1.9080, ..., 1.9428, 1.9254, 1.9080],
 ...,
 [ 1.9951, 1.9951, 2.0125, ..., 1.4722, 1.2108, 1.1759],
 [ 1.9951, 1.9951, 2.0125, ..., 1.4548, 1.1759, 1.2108],
 [ 1.9951, 1.9777, 1.9951, ..., 1.2457, 1.3851, 1.4897]]],

[[[-0.8849, -0.8849, -0.8678, ..., -0.7479, -0.7308, -0.7308],
 [-0.8849, -0.8678, -0.8507, ..., -0.7479, -0.7308, -0.7308],
 [-0.8507, -0.8507, -0.8335, ..., -0.7479, -0.7308, -0.7308],
 ...,
 [-0.9534, -0.9534, -0.9363, ..., -0.8678, -0.8678, -0.8678],
 [-0.8507, -0.8849, -0.8335, ..., -0.8678, -0.8849, -0.8849],
 [ 0.1426, -0.6794, -0.1314, ..., -0.8678, -0.8849, -0.8849]],

[[ -0.2500, -0.2325, -0.2150, ..., -0.0399, -0.0224, -0.0224],
 [ -0.2500, -0.2150, -0.1975, ..., -0.0399, -0.0224, -0.0224],
 [ -0.2150, -0.1975, -0.1800, ..., -0.0399, -0.0224, -0.0224],
 ...,
 [ -0.3725, -0.3725, -0.3375, ..., -0.1975, -0.1975, -0.1975],
 [ -0.3550, -0.3725, -0.3200, ..., -0.1975, -0.1800, -0.1975],
 [ 0.6078, -0.2325, 0.3627, ..., -0.2150, -0.1975, -0.2150]],

```

```

[[[-1.1944, -1.1770, -1.1596, ..., -1.1073, -1.0898, -1.0898],
 [-1.1770, -1.1596, -1.1421, ..., -1.1073, -1.0898, -1.0898],
 [-1.1596, -1.1421, -1.1247, ..., -1.1073, -1.0898, -1.0898],
 ...,
 [-1.1770, -1.2119, -1.2293, ..., -1.2816, -1.2990, -1.2990],
 [-0.9678, -1.0376, -1.0550, ..., -1.2816, -1.2641, -1.2293],
 [ 0.1476, -0.7238, -0.2358, ..., -1.2293, -1.1944, -1.1770]]],

[[[ 0.8276, 0.7933, 0.7591, ..., 0.6049, 0.5878, 0.5878],
 [ 0.8276, 0.7933, 0.7591, ..., 0.6049, 0.5878, 0.5878],
 [ 0.8276, 0.7762, 0.7419, ..., 0.6221, 0.5878, 0.5707],
 ...,
 [ 0.7419, 0.7248, 0.7248, ..., 0.1083, 0.1426, 0.1768],
 [ 0.7591, 0.7591, 0.7762, ..., 0.1254, 0.1597, 0.2111],
 [ 0.7419, 0.7591, 0.7762, ..., 0.1426, 0.1939, 0.2453]]],

[[ 0.8704, 0.8529, 0.8354, ..., 1.0455, 1.0630, 1.0630],
 [ 0.8880, 0.8529, 0.8354, ..., 1.0455, 1.0630, 1.0630],
 [ 0.8880, 0.8529, 0.8179, ..., 1.0455, 1.0280, 1.0280],
 ...,
 [ 0.8529, 0.8354, 0.8529, ..., 0.0826, 0.1176, 0.1352],
 [ 0.8880, 0.8880, 0.9055, ..., 0.1001, 0.1176, 0.1352],
 [ 0.8704, 0.8880, 0.9055, ..., 0.1176, 0.1352, 0.1702]]],

[[[-0.3753, -0.3404, -0.2532, ..., -0.0964, -0.0964, -0.0964],
 [-0.4101, -0.3578, -0.2532, ..., -0.0964, -0.0964, -0.0964],
 [-0.4450, -0.3927, -0.2707, ..., -0.0964, -0.1138, -0.1312],
 ...,
 [ 0.1651, 0.1476, 0.1302, ..., -0.6715, -0.6193, -0.5670],
 [ 0.1651, 0.1651, 0.1476, ..., -0.6367, -0.5844, -0.5321],
 [ 0.1476, 0.1651, 0.1476, ..., -0.6193, -0.5670, -0.4973]]]]), tensor([1
1, 6, 71, 74, 51, 63, 59, 60, 8, 56, 50, 56, 7, 53, 6, 3, 36, 31,
1, 15, 47, 56, 2, 58, 18, 5, 1, 20, 42, 52, 70, 6])]

```

```

In [17]: import torch
import torch.nn.functional as F
import torch.nn as nn
import matplotlib.pyplot as plt

# Determine the number of unique labels in the **full training dataset**
num_classes = full_train_dataset.data['label'].nunique() # Use full_train_dataset to
print(f"Number of classes in the dataset: {num_classes}")

# Define the models with the correct number of classes
model1 = CNNModel(num_classes=num_classes)
model2 = MLPModel(num_classes=num_classes)
model3 = RNNModel(num_classes=num_classes, input_size=64, hidden_size=128, num_layers=

# List of models and their names for tracking
models = [model1, model2, model3]
model_names = ["Convolutional Neural Network (CNN)", "Multilayer Perceptron (MLP)", "R

# Define the learning rates to be tested

learning_rates = [0.001, 0.0001, 0.00001]

# Assuming `train_loader`, `val_loader`, `criterion`, and `device` are already defined

```

```

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
criterion = nn.CrossEntropyLoss() # Assuming a classification problem

# Function to train and visualize each model separately with early stopping
def train_and_visualize_models_separately(models, model_names, train_loader, val_loader):
    for model_index, (model, model_name) in enumerate(zip(models, model_names)):
        print(f"\n\nStarting Training for {model_name}")

        # Create a new figure for each model's visualization
        plt.figure(figsize=(12, 8))

        for lr in learning_rates:
            print(f"\nTraining {model_name} with Learning Rate: {lr}")

            # Move the model to the correct device (GPU/CPU)
            model.to(device)

            # Initialize optimizer with the current Learning rate
            optimizer = torch.optim.Adam(model.parameters(), lr=lr)

            # Variables for early stopping
            train_losses = []
            val_losses = []
            best_val_loss = float("inf")
            patience_counter = 0

            for epoch in range(num_epochs):
                # Training phase
                model.train()
                running_loss = 0.0
                for batch in train_loader:
                    inputs, labels = batch[0], batch[1]
                    inputs, labels = inputs.to(device), labels.to(device)

                    optimizer.zero_grad()
                    outputs = model(inputs)
                    loss = criterion(outputs, labels)
                    loss.backward()
                    optimizer.step()

                    running_loss += loss.item() * inputs.size(0)

                epoch_train_loss = running_loss / len(train_loader.dataset)
                train_losses.append(epoch_train_loss)

                # Validation phase
                model.eval()
                running_val_loss = 0.0
                with torch.no_grad():
                    for batch in val_loader:
                        inputs, labels = batch[0], batch[1]
                        inputs, labels = inputs.to(device), labels.to(device)
                        outputs = model(inputs)
                        loss = criterion(outputs, labels)
                        running_val_loss += loss.item() * inputs.size(0)

                epoch_val_loss = running_val_loss / len(val_loader.dataset)
                val_losses.append(epoch_val_loss)

            print(f"Epoch {epoch + 1}/{num_epochs}, Train Loss: {epoch_train_loss:}

```

```

        # Early stopping check
        if (best_val_loss - epoch_val_loss) < improvement_threshold:
            patience_counter += 1
            print(f"No significant improvement in validation loss ( $\Delta$  < {improvement_threshold})")
        else:
            best_val_loss = epoch_val_loss
            patience_counter = 0

        if patience_counter >= patience:
            print("Early stopping triggered")
            break

    # Plot the training and validation losses for this learning rate
    plt.plot(train_losses, label=f'Train Loss (LR={lr})')
    plt.plot(val_losses, label=f'Val Loss (LR={lr})')

    # Print the final training and validation loss for this model and learning rate
    print(f"Final Training Loss for {model_name} with LR={lr}: {train_losses[-1]}")
    print(f"Final Validation Loss for {model_name} with LR={lr}: {val_losses[-1]}")

    # Reset the model parameters to re-train for each learning rate
    model.apply(lambda layer: layer.reset_parameters() if hasattr(layer, 'reset_parameters'))

    # Set up plot labels and titles for the current model
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.title(f'{model_name} - Training and Validation Loss with Different Learning Rates')
    plt.legend()
    plt.grid(True)
    plt.show()

# Call the function to train and visualize each model separately with early stopping
train_and_visualize_models_separately(models, model_names, train_loader, val_loader, patience, improvement_threshold)

```

Number of classes in the dataset: 75

Starting Training for Convolutional Neural Network (CNN)

Training Convolutional Neural Network (CNN) with Learning Rate: 0.001

Epoch 1/5, Train Loss: 3.3322, Val Loss: 2.3975

Epoch 2/5, Train Loss: 1.9967, Val Loss: 1.4689

Epoch 3/5, Train Loss: 1.3312, Val Loss: 0.8799

Epoch 4/5, Train Loss: 0.8337, Val Loss: 0.4582

Epoch 5/5, Train Loss: 0.4589, Val Loss: 0.2537

No significant improvement in validation loss ($\Delta < 0.25$), patience counter: 1/2

Final Training Loss for Convolutional Neural Network (CNN) with LR=0.001: 0.4589

Final Validation Loss for Convolutional Neural Network (CNN) with LR=0.001: 0.2537

Training Convolutional Neural Network (CNN) with Learning Rate: 0.0001

Epoch 1/5, Train Loss: 4.0410, Val Loss: 3.4540

Epoch 2/5, Train Loss: 3.0810, Val Loss: 2.6019

Epoch 3/5, Train Loss: 2.4578, Val Loss: 2.1595

Epoch 4/5, Train Loss: 2.0604, Val Loss: 1.8515

Epoch 5/5, Train Loss: 1.7774, Val Loss: 1.5495

Final Training Loss for Convolutional Neural Network (CNN) with LR=0.0001: 1.7774

Final Validation Loss for Convolutional Neural Network (CNN) with LR=0.0001: 1.5495

Training Convolutional Neural Network (CNN) with Learning Rate: 1e-05

Epoch 1/5, Train Loss: 4.2893, Val Loss: 4.2336

Epoch 2/5, Train Loss: 4.1357, Val Loss: 3.9834

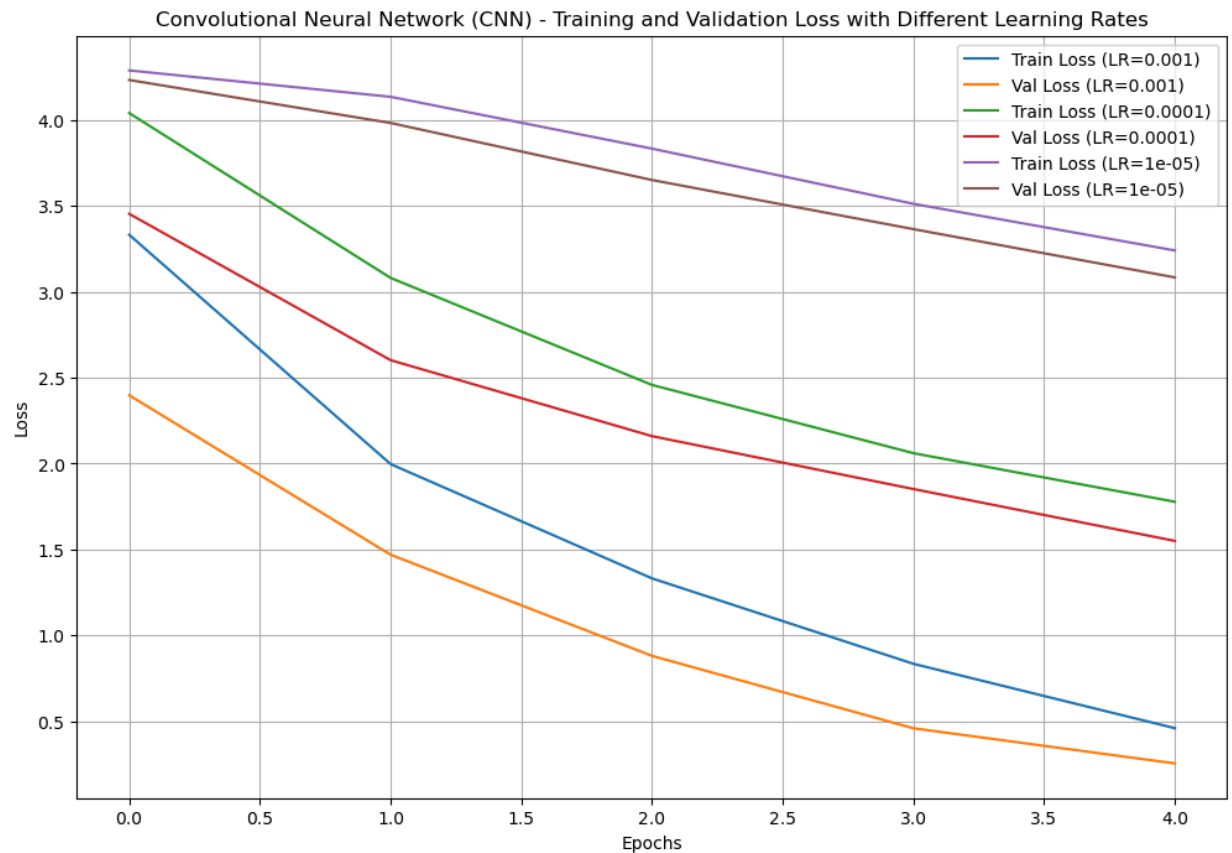
Epoch 3/5, Train Loss: 3.8338, Val Loss: 3.6512

Epoch 4/5, Train Loss: 3.5124, Val Loss: 3.3651

Epoch 5/5, Train Loss: 3.2410, Val Loss: 3.0839

Final Training Loss for Convolutional Neural Network (CNN) with LR=1e-05: 3.2410

Final Validation Loss for Convolutional Neural Network (CNN) with LR=1e-05: 3.0839



Starting Training for Multilayer Perceptron (MLP)

Training Multilayer Perceptron (MLP) with Learning Rate: 0.001

Epoch 1/5, Train Loss: 4.3891, Val Loss: 3.8361

Epoch 2/5, Train Loss: 3.6989, Val Loss: 3.4133

Epoch 3/5, Train Loss: 3.4193, Val Loss: 3.2540

No significant improvement in validation loss ($\Delta < 0.25$), patience counter: 1/2

Epoch 4/5, Train Loss: 3.2125, Val Loss: 3.1147

Epoch 5/5, Train Loss: 3.0438, Val Loss: 2.9266

No significant improvement in validation loss ($\Delta < 0.25$), patience counter: 1/2

Final Training Loss for Multilayer Perceptron (MLP) with LR=0.001: 3.0438

Final Validation Loss for Multilayer Perceptron (MLP) with LR=0.001: 2.9266

Training Multilayer Perceptron (MLP) with Learning Rate: 0.0001

Epoch 1/5, Train Loss: 3.5789, Val Loss: 2.4831

Epoch 2/5, Train Loss: 2.4345, Val Loss: 1.6408

Epoch 3/5, Train Loss: 1.6679, Val Loss: 1.0655

Epoch 4/5, Train Loss: 1.1076, Val Loss: 0.7070

Epoch 5/5, Train Loss: 0.7229, Val Loss: 0.4470

Final Training Loss for Multilayer Perceptron (MLP) with LR=0.0001: 0.7229

Final Validation Loss for Multilayer Perceptron (MLP) with LR=0.0001: 0.4470

Training Multilayer Perceptron (MLP) with Learning Rate: 1e-05

Epoch 1/5, Train Loss: 3.8955, Val Loss: 3.4079

Epoch 2/5, Train Loss: 3.2659, Val Loss: 2.9060

Epoch 3/5, Train Loss: 2.8477, Val Loss: 2.5549

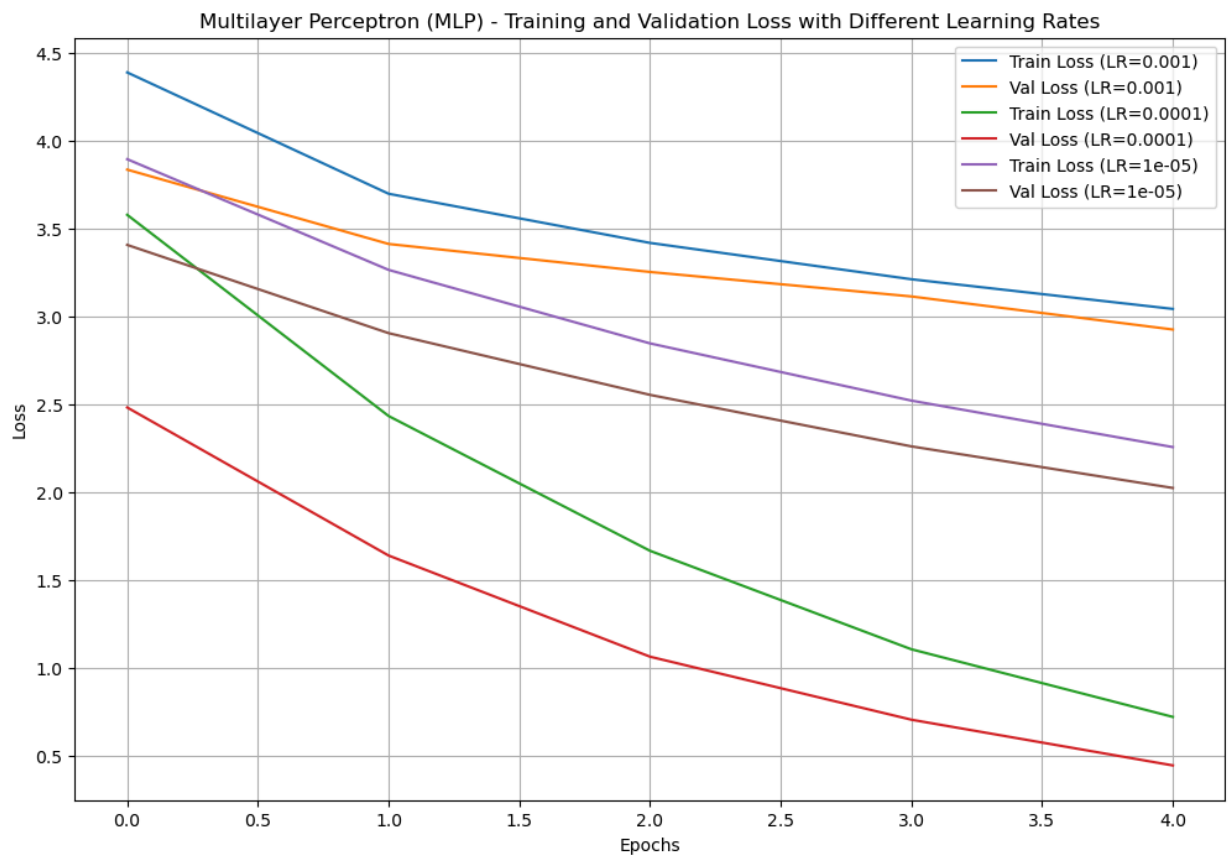
Epoch 4/5, Train Loss: 2.5222, Val Loss: 2.2618

Epoch 5/5, Train Loss: 2.2578, Val Loss: 2.0253

No significant improvement in validation loss ($\Delta < 0.25$), patience counter: 1/2

Final Training Loss for Multilayer Perceptron (MLP) with LR=1e-05: 2.2578

Final Validation Loss for Multilayer Perceptron (MLP) with LR=1e-05: 2.0253



Starting Training for Recurrent Neural Network (RNN)

Training Recurrent Neural Network (RNN) with Learning Rate: 0.001

Epoch 1/5, Train Loss: 4.2826, Val Loss: 4.2424

Epoch 2/5, Train Loss: 4.2390, Val Loss: 4.2162

No significant improvement in validation loss ($\Delta < 0.25$), patience counter: 1/2

Epoch 3/5, Train Loss: 4.2171, Val Loss: 4.1819

No significant improvement in validation loss ($\Delta < 0.25$), patience counter: 2/2

Early stopping triggered

Final Training Loss for Recurrent Neural Network (RNN) with LR=0.001: 4.2171

Final Validation Loss for Recurrent Neural Network (RNN) with LR=0.001: 4.1819

Training Recurrent Neural Network (RNN) with Learning Rate: 0.0001

Epoch 1/5, Train Loss: 4.3068, Val Loss: 4.2899

Epoch 2/5, Train Loss: 4.2725, Val Loss: 4.2689

No significant improvement in validation loss ($\Delta < 0.25$), patience counter: 1/2

Epoch 3/5, Train Loss: 4.2551, Val Loss: 4.2564

No significant improvement in validation loss ($\Delta < 0.25$), patience counter: 2/2

Early stopping triggered

Final Training Loss for Recurrent Neural Network (RNN) with LR=0.0001: 4.2551

Final Validation Loss for Recurrent Neural Network (RNN) with LR=0.0001: 4.2564

Training Recurrent Neural Network (RNN) with Learning Rate: 1e-05

Epoch 1/5, Train Loss: 4.3174, Val Loss: 4.3156

Epoch 2/5, Train Loss: 4.3150, Val Loss: 4.3135

No significant improvement in validation loss ($\Delta < 0.25$), patience counter: 1/2

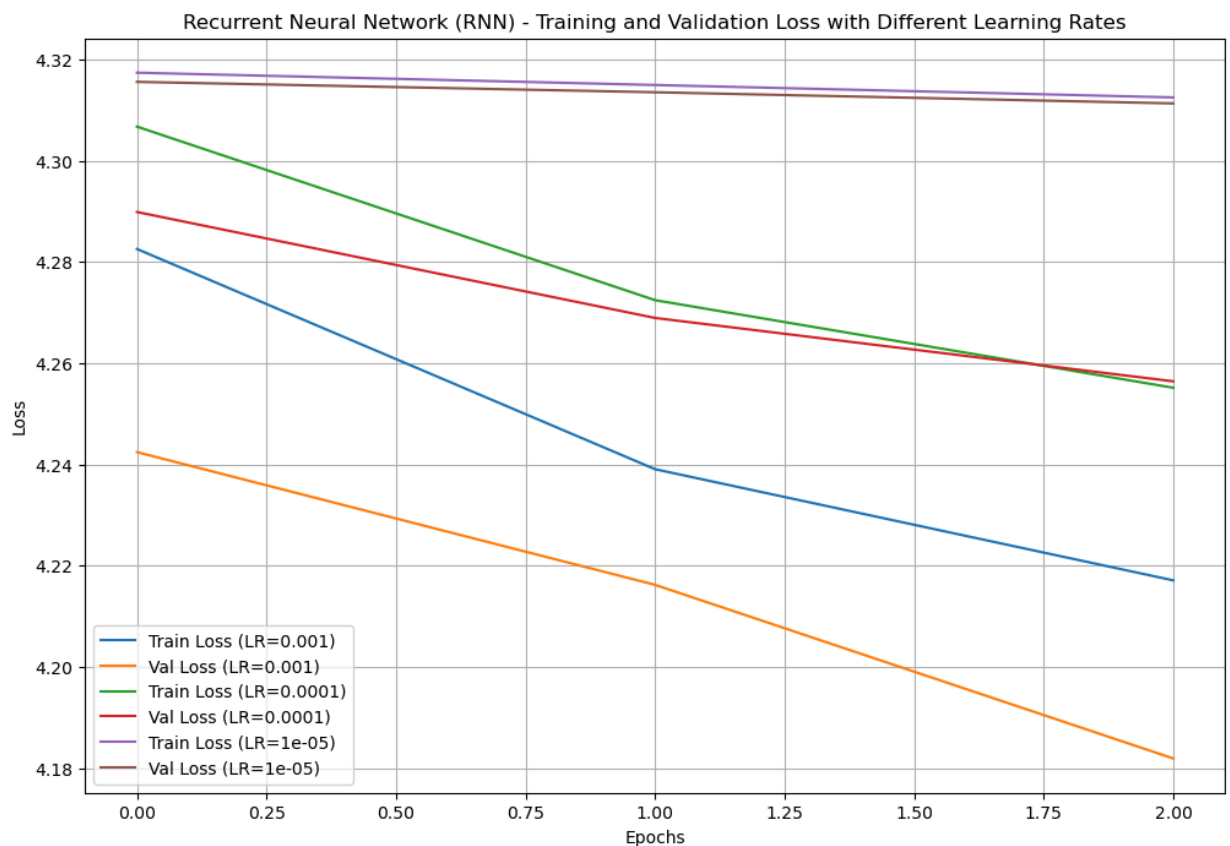
Epoch 3/5, Train Loss: 4.3125, Val Loss: 4.3113

No significant improvement in validation loss ($\Delta < 0.25$), patience counter: 2/2

Early stopping triggered

Final Training Loss for Recurrent Neural Network (RNN) with LR=1e-05: 4.3125

Final Validation Loss for Recurrent Neural Network (RNN) with LR=1e-05: 4.3113




```
In [18]: #2E Evaluate models with suitable metrics and explain the results.
```

```
In [25]: import torch
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, c
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Function to evaluate the model on the validation or test set
def evaluate_model(model, dataloader, criterion, device):
    model.eval() # Set the model to evaluation mode
    true_labels = []
    pred_labels = []
    total_loss = 0.0

    with torch.no_grad(): # Disable gradient computation for evaluation
        for inputs, labels in dataloader:
            inputs, labels = inputs.to(device), labels.to(device)

            # Forward pass
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * inputs.size(0)

            # Get predictions
            _, preds = torch.max(outputs, 1) # Get class index with highest score
            true_labels.extend(labels.cpu().numpy())
            pred_labels.extend(preds.cpu().numpy())

    # Calculate average loss
    avg_loss = total_loss / len(dataloader.dataset)

    # Calculate metrics
    accuracy = accuracy_score(true_labels, pred_labels)
    precision = precision_score(true_labels, pred_labels, average='weighted') # Weighted
    recall = recall_score(true_labels, pred_labels, average='weighted')
    f1 = f1_score(true_labels, pred_labels, average='weighted')

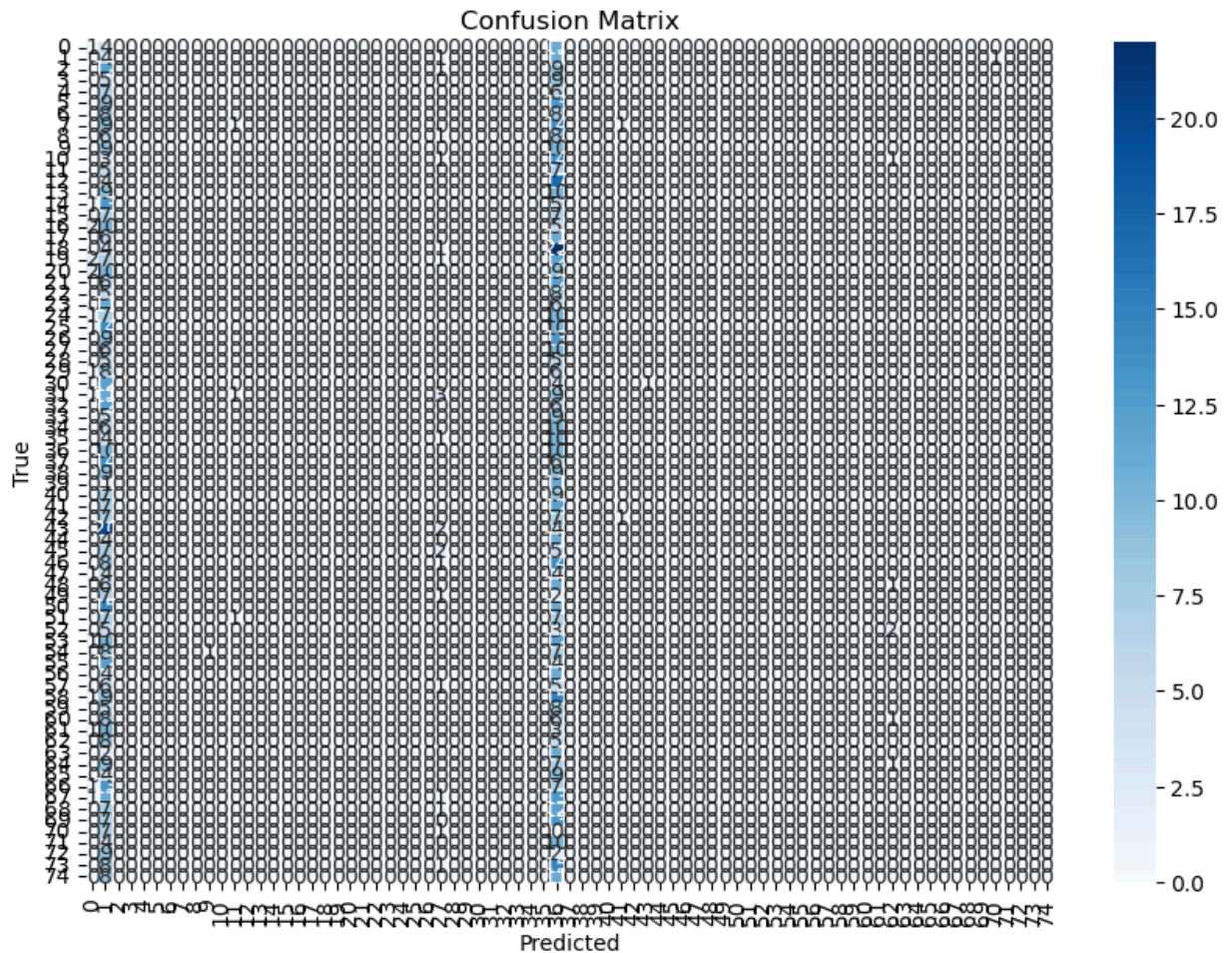
    # Print evaluation results
    print(f"Loss: {avg_loss:.4f}, Accuracy: {accuracy:.4f}, Precision: {precision:.4f}")

    # Confusion matrix
    cm = confusion_matrix(true_labels, pred_labels)

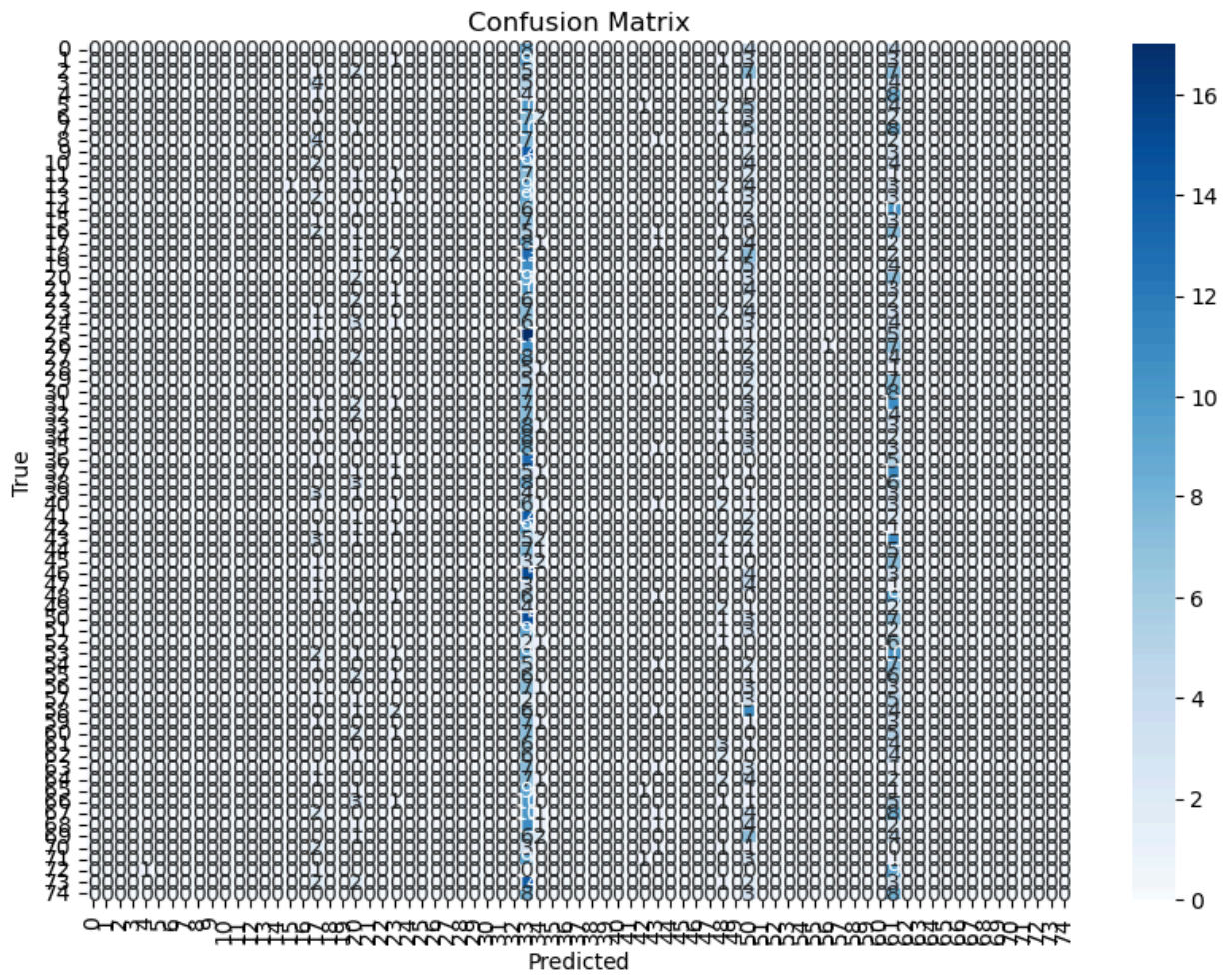
    # Plot confusion matrix
    plt.figure(figsize=(10, 7))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(true_labels),
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.title('Confusion Matrix')
    plt.show()

evaluate_model(model1, val_loader, criterion, device)
evaluate_model(model2, val_loader, criterion, device)
evaluate_model(model3, val_loader, criterion, device)
```

F:\Programs\Anaconda\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.
 _warn_prf(average, modifier, msg_start, len(result))
 Loss: 4.3201, Accuracy: 0.0115, Precision: 0.0010, Recall: 0.0115, F1-Score: 0.0013



F:\Programs\Anaconda\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.
 _warn_prf(average, modifier, msg_start, len(result))
 Loss: 4.3170, Accuracy: 0.0123, Precision: 0.0058, Recall: 0.0123, F1-Score: 0.0063



In [20]: *#For the ‘best-performing model’, you are required to repeat modeling experiments at least 5 times with different random trials of data sampling. Statistical metrics (e.g., mean, standard deviation, 95% Confidence Interval, etc.) across multiple trials are required to provide evidence on the model’s robustness and reproducibility*

In []:

```
In [21]: import numpy as np
import scipy.stats as st

# Set random seeds for reproducibility
def set_random_seed(seed):
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    np.random.seed(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

# Function to repeat experiments and collect losses
def repeat_experiments(model_class, num_trials=5, num_epochs=5, learning_rate=0.001):
    train_losses_trials = []
    val_losses_trials = []

    for trial in range(num_trials):
        print(f"\nTrial {trial + 1}/{num_trials}")
        set_random_seed(trial) # Set different seed for each trial
```

```

# Initialize model and optimizer
model = model_class().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

# Train the model
train_losses, val_losses = train_model(model, train_loader, val_loader, criterion)

# Collect final epoch losses
train_losses_trials.append(train_losses[-1]) # Store final training Loss
val_losses_trials.append(val_losses[-1])     # Store final validation Loss

return train_losses_trials, val_losses_trials

# Run 5 trials for CNN with LR=0.001
train_losses_trials, val_losses_trials = repeat_experiments(CNNModel, num_trials=5, num_epochs=10)

# Calculate mean, standard deviation, and 95% confidence interval
def calculate_statistics(losses):
    mean_loss = np.mean(losses)
    std_loss = np.std(losses)
    conf_interval = st.t.interval(0.95, len(losses)-1, loc=mean_loss, scale=st.sem(losses))

    return mean_loss, std_loss, conf_interval

# Calculate statistics for training and validation losses
mean_train_loss, std_train_loss, ci_train_loss = calculate_statistics(train_losses_trials)
mean_val_loss, std_val_loss, ci_val_loss = calculate_statistics(val_losses_trials)

# Print out the results
print(f"Training Loss: Mean = {mean_train_loss:.4f}, Std = {std_train_loss:.4f}, 95% CI = {ci_train_loss}")
print(f"Validation Loss: Mean = {mean_val_loss:.4f}, Std = {std_val_loss:.4f}, 95% CI = {ci_val_loss}")

```

Trial 1/5
Epoch 1/5, Train Loss: 3.3955, Val Loss: 2.2132
Epoch 2/5, Train Loss: 1.9036, Val Loss: 1.2483
Epoch 3/5, Train Loss: 1.1462, Val Loss: 0.6655
Epoch 4/5, Train Loss: 0.6362, Val Loss: 0.3084
Epoch 5/5, Train Loss: 0.3156, Val Loss: 0.1540

Trial 2/5
Epoch 1/5, Train Loss: 3.3012, Val Loss: 2.1808
Epoch 2/5, Train Loss: 1.9621, Val Loss: 1.3332
Epoch 3/5, Train Loss: 1.2768, Val Loss: 0.8063
Epoch 4/5, Train Loss: 0.8339, Val Loss: 0.4437
Epoch 5/5, Train Loss: 0.4672, Val Loss: 0.2527

Trial 3/5
Epoch 1/5, Train Loss: 3.3283, Val Loss: 2.1409
Epoch 2/5, Train Loss: 1.8730, Val Loss: 1.3385
Epoch 3/5, Train Loss: 1.2582, Val Loss: 0.8009
Epoch 4/5, Train Loss: 0.7886, Val Loss: 0.4093
Epoch 5/5, Train Loss: 0.4417, Val Loss: 0.2263

Trial 4/5
Epoch 1/5, Train Loss: 3.4537, Val Loss: 2.4286
Epoch 2/5, Train Loss: 2.0387, Val Loss: 1.4530
Epoch 3/5, Train Loss: 1.3332, Val Loss: 0.9828
Epoch 4/5, Train Loss: 0.8758, Val Loss: 0.5535
Epoch 5/5, Train Loss: 0.4741, Val Loss: 0.2701

Trial 5/5
Epoch 1/5, Train Loss: 3.3524, Val Loss: 2.0653
Epoch 2/5, Train Loss: 1.7405, Val Loss: 1.1112
Epoch 3/5, Train Loss: 0.9837, Val Loss: 0.5274
Epoch 4/5, Train Loss: 0.4770, Val Loss: 0.2699
Epoch 5/5, Train Loss: 0.1998, Val Loss: 0.1760
Training Loss: Mean = 0.3797, Std = 0.1067, 95% CI = (0.2316179822297403, 0.5277677304909296)
Validation Loss: Mean = 0.2158, Std = 0.0443, 95% CI = (0.15429015934391604, 0.27739509987906824)

In [27]: `torch.save(model.state_dict(), 'F:\\Jordan\\Documents\\Github\\SD7502-Project2-2220115`

In [22]: `#Web application implementation`

In [23]: `!pip install ipywidgets voila`

Requirement already satisfied: ipywidgets in f:\programs\anaconda\lib\site-packages (8.0.4)

Collecting voila

Downloading voila-0.5.7-py3-none-any.whl (3.9 MB)

```
0.0/3.9 MB ? eta -:--:--
----- 0.8/3.9 MB 16.3 MB/s eta 0:00:01
----- 1.0/3.9 MB 22.0 MB/s eta 0:00:01
----- 1.0/3.9 MB 22.0 MB/s eta 0:00:01
----- 1.0/3.9 MB 22.0 MB/s eta 0:00:01
----- 1.0/3.9 MB 22.0 MB/s eta 0:00:01
----- 1.4/3.9 MB 5.0 MB/s eta 0:00:01
----- 2.1/3.9 MB 7.0 MB/s eta 0:00:01
----- 2.1/3.9 MB 7.0 MB/s eta 0:00:01
----- 2.1/3.9 MB 7.0 MB/s eta 0:00:01
----- 2.1/3.9 MB 7.0 MB/s eta 0:00:01
----- 2.5/3.9 MB 4.9 MB/s eta 0:00:01
----- 3.1/3.9 MB 5.9 MB/s eta 0:00:01
----- 3.1/3.9 MB 5.9 MB/s eta 0:00:01
----- 3.1/3.9 MB 5.9 MB/s eta 0:00:01
----- 3.1/3.9 MB 5.9 MB/s eta 0:00:01
----- 3.9/3.9 MB 5.2 MB/s eta 0:00:01
----- 3.9/3.9 MB 5.1 MB/s eta 0:00:00
```

Requirement already satisfied: ipykernel>=4.5.1 in f:\programs\anaconda\lib\site-packages (from ipywidgets) (6.19.2)

Requirement already satisfied: ipython>=6.1.0 in f:\programs\anaconda\lib\site-packages (from ipywidgets) (8.12.0)

Requirement already satisfied: traitlets>=4.3.1 in f:\programs\anaconda\lib\site-packages (from ipywidgets) (5.7.1)

Requirement already satisfied: widgetsnbextension~=4.0 in f:\programs\anaconda\lib\site-packages (from ipywidgets) (4.0.5)

Requirement already satisfied: jupyterlab-widgets~=3.0 in f:\programs\anaconda\lib\site-packages (from ipywidgets) (3.0.5)

Requirement already satisfied: jupyter-client<9,>=7.4.4 in f:\programs\anaconda\lib\site-packages (from voila) (8.1.0)

Requirement already satisfied: jupyter-core>=4.11.0 in f:\programs\anaconda\lib\site-packages (from voila) (5.3.0)

Requirement already satisfied: jupyter-server<3,>=2.0.0 in f:\programs\anaconda\lib\site-packages (from voila) (2.5.0)

Requirement already satisfied: jupyterlab-server<3,>=2.3.0 in f:\programs\anaconda\lib\site-packages (from voila) (2.22.0)

Requirement already satisfied: nbclient>=0.4.0 in f:\programs\anaconda\lib\site-packages (from voila) (0.5.13)

Requirement already satisfied: nbconvert<8,>=6.4.5 in f:\programs\anaconda\lib\site-packages (from voila) (6.5.4)

Collecting websockets>=9.0 (from voila)

Downloading websockets-13.1-cp311-cp311-win_amd64.whl (159 kB)

```
0.0/159.1 kB ? eta -:--:--
----- 159.1/159.1 kB 9.3 MB/s eta 0:00:00
```

Requirement already satisfied: comm>=0.1.1 in f:\programs\anaconda\lib\site-packages (from ipykernel>=4.5.1->ipywidgets) (0.1.2)

Requirement already satisfied: debugpy>=1.0 in f:\programs\anaconda\lib\site-packages (from ipykernel>=4.5.1->ipywidgets) (1.5.1)

Requirement already satisfied: matplotlib-inline>=0.1 in f:\programs\anaconda\lib\site-packages (from ipykernel>=4.5.1->ipywidgets) (0.1.6)

Requirement already satisfied: nest-asyncio in f:\programs\anaconda\lib\site-packages (from ipykernel>=4.5.1->ipywidgets) (1.5.6)

Requirement already satisfied: packaging in f:\programs\anaconda\lib\site-packages (from ipykernel>=4.5.1->ipywidgets) (23.0)

Requirement already satisfied: psutil in f:\programs\anaconda\lib\site-packages (from ipykernel>=4.5.1->ipywidgets) (5.9.0)

Requirement already satisfied: pyzmq>=17 in f:\programs\anaconda\lib\site-packages (from ipykernel>=4.5.1->ipywidgets) (25.1.0)

Requirement already satisfied: tornado>=6.1 in f:\programs\anaconda\lib\site-packages (from ipykernel>=4.5.1->ipywidgets) (6.2)

Requirement already satisfied: backcall in f:\programs\anaconda\lib\site-packages (from ipython>=6.1.0->ipywidgets) (0.2.0)

Requirement already satisfied: decorator in f:\programs\anaconda\lib\site-packages (from ipython>=6.1.0->ipywidgets) (5.1.1)

Requirement already satisfied: jedi>=0.16 in f:\programs\anaconda\lib\site-packages (from ipython>=6.1.0->ipywidgets) (0.18.1)

Requirement already satisfied: pickleshare in f:\programs\anaconda\lib\site-packages (from ipython>=6.1.0->ipywidgets) (0.7.5)

Requirement already satisfied: prompt-toolkit!=3.0.37,<3.1.0,>=3.0.30 in f:\programs\anaconda\lib\site-packages (from ipython>=6.1.0->ipywidgets) (3.0.36)

Requirement already satisfied: pygments>=2.4.0 in f:\programs\anaconda\lib\site-packages (from ipython>=6.1.0->ipywidgets) (2.15.1)

Requirement already satisfied: stack-data in f:\programs\anaconda\lib\site-packages (from ipython>=6.1.0->ipywidgets) (0.2.0)

Requirement already satisfied: colorama in f:\programs\anaconda\lib\site-packages (from ipython>=6.1.0->ipywidgets) (0.4.6)

Requirement already satisfied: python-dateutil>=2.8.2 in f:\programs\anaconda\lib\site-packages (from jupyter-client<9,>=7.4.4->voila) (2.8.2)

Requirement already satisfied: platformdirs>=2.5 in f:\programs\anaconda\lib\site-packages (from jupyter-core>=4.11.0->voila) (2.5.2)

Requirement already satisfied: pywin32>=300 in f:\programs\anaconda\lib\site-packages (from jupyter-core>=4.11.0->voila) (305.1)

Requirement already satisfied: anyio>=3.1.0 in f:\programs\anaconda\lib\site-packages (from jupyter-server<3,>=2.0.0->voila) (3.5.0)

Requirement already satisfied: argon2-cffi in f:\programs\anaconda\lib\site-packages (from jupyter-server<3,>=2.0.0->voila) (21.3.0)

Requirement already satisfied: jinja2 in c:\users\jordan\appdata\roaming\python\python311\site-packages (from jupyter-server<3,>=2.0.0->voila) (3.1.4)

Requirement already satisfied: jupyter-events>=0.4.0 in f:\programs\anaconda\lib\site-packages (from jupyter-server<3,>=2.0.0->voila) (0.6.3)

Requirement already satisfied: jupyter-server-terminals in f:\programs\anaconda\lib\site-packages (from jupyter-server<3,>=2.0.0->voila) (0.4.4)

Requirement already satisfied: nbformat>=5.3.0 in f:\programs\anaconda\lib\site-packages (from jupyter-server<3,>=2.0.0->voila) (5.7.0)

Requirement already satisfied: prometheus-client in f:\programs\anaconda\lib\site-packages (from jupyter-server<3,>=2.0.0->voila) (0.14.1)

Requirement already satisfied: pywinpty in f:\programs\anaconda\lib\site-packages (from jupyter-server<3,>=2.0.0->voila) (2.0.10)

Requirement already satisfied: send2trash in f:\programs\anaconda\lib\site-packages (from jupyter-server<3,>=2.0.0->voila) (1.8.0)

Requirement already satisfied: terminado>=0.8.3 in f:\programs\anaconda\lib\site-packages (from jupyter-server<3,>=2.0.0->voila) (0.17.1)

Requirement already satisfied: websocket-client in f:\programs\anaconda\lib\site-packages (from jupyter-server<3,>=2.0.0->voila) (0.58.0)

Requirement already satisfied: babel>=2.10 in f:\programs\anaconda\lib\site-packages (from jupyterlab-server<3,>=2.3.0->voila) (2.11.0)

Requirement already satisfied: json5>=0.9.0 in f:\programs\anaconda\lib\site-packages (from jupyterlab-server<3,>=2.3.0->voila) (0.9.6)

Requirement already satisfied: jsonschema>=4.17.3 in f:\programs\anaconda\lib\site-packages (from jupyterlab-server<3,>=2.3.0->voila) (4.17.3)

Requirement already satisfied: requests>=2.28 in f:\programs\anaconda\lib\site-packages (from jupyterlab-server<3,>=2.3.0->voila) (2.32.3)

Requirement already satisfied: lxml in f:\programs\anaconda\lib\site-packages (from nbconvert<8,>=6.4.5->voila) (4.9.2)

Requirement already satisfied: beautifulsoup4 in f:\programs\anaconda\lib\site-packages (from nbconvert<8,>=6.4.5->voila) (4.12.2)

Requirement already satisfied: bleach in f:\programs\anaconda\lib\site-packages (from nbconvert<8,>=6.4.5->voila) (4.1.0)

Requirement already satisfied: defusedxml in f:\programs\anaconda\lib\site-packages (from nbconvert<8,>=6.4.5->voila) (0.7.1)

Requirement already satisfied: entrypoints>=0.2.2 in f:\programs\anaconda\lib\site-packages (from nbconvert<8,>=6.4.5->voila) (0.4)

Requirement already satisfied: jupyterlab-pygments in f:\programs\anaconda\lib\site-packages (from nbconvert<8,>=6.4.5->voila) (0.1.2)

Requirement already satisfied: MarkupSafe>=2.0 in f:\programs\anaconda\lib\site-packages (from nbconvert<8,>=6.4.5->voila) (3.0.1)

Requirement already satisfied: mistune<2,>=0.8.1 in f:\programs\anaconda\lib\site-packages (from nbconvert<8,>=6.4.5->voila) (0.8.4)

Requirement already satisfied: pandocfilters>=1.4.1 in f:\programs\anaconda\lib\site-packages (from nbconvert<8,>=6.4.5->voila) (1.5.0)

Requirement already satisfied: tinycss2 in f:\programs\anaconda\lib\site-packages (from nbconvert<8,>=6.4.5->voila) (1.2.1)

Requirement already satisfied: idna>=2.8 in f:\programs\anaconda\lib\site-packages (from anyio>=3.1.0->jupyter-server<3,>=2.0.0->voila) (3.4)

Requirement already satisfied: sniffio>=1.1 in f:\programs\anaconda\lib\site-packages (from anyio>=3.1.0->jupyter-server<3,>=2.0.0->voila) (1.2.0)

Requirement already satisfied: pytz>=2015.7 in f:\programs\anaconda\lib\site-packages (from babel>=2.10->jupyterlab-server<3,>=2.3.0->voila) (2022.7)

Requirement already satisfied: parso<0.9.0,>=0.8.0 in f:\programs\anaconda\lib\site-packages (from jedi>=0.16->ipython>=6.1.0->ipywidgets) (0.8.3)

Requirement already satisfied: attrs>=17.4.0 in f:\programs\anaconda\lib\site-packages (from jsonschema>=4.17.3->jupyterlab-server<3,>=2.3.0->voila) (22.1.0)

Requirement already satisfied: pyparsing!=0.17.0,!0.17.1,!0.17.2,>=0.14.0 in f:\programs\anaconda\lib\site-packages (from jsonschema>=4.17.3->jupyterlab-server<3,>=2.3.0->voila) (0.18.0)

Requirement already satisfied: python-json-logger>=2.0.4 in f:\programs\anaconda\lib\site-packages (from jupyter-events>=0.4.0->jupyter-server<3,>=2.0.0->voila) (2.0.7)

Requirement already satisfied: pyyaml>=5.3 in f:\programs\anaconda\lib\site-packages (from jupyter-events>=0.4.0->jupyter-server<3,>=2.0.0->voila) (6.0)

Requirement already satisfied: rfc3339-validator in f:\programs\anaconda\lib\site-packages (from jupyter-events>=0.4.0->jupyter-server<3,>=2.0.0->voila) (0.1.4)

Requirement already satisfied: rfc3986-validator>=0.1.1 in f:\programs\anaconda\lib\site-packages (from jupyter-events>=0.4.0->jupyter-server<3,>=2.0.0->voila) (0.1.1)

Requirement already satisfied: fastjsonschema in f:\programs\anaconda\lib\site-packages (from nbformat>=5.3.0->jupyter-server<3,>=2.0.0->voila) (2.16.2)

Requirement already satisfied: wcwidth in f:\programs\anaconda\lib\site-packages (from prompt-toolkit!=3.0.37,<3.1.0,>=3.0.30->ipython>=6.1.0->ipywidgets) (0.2.5)

Requirement already satisfied: six>=1.5 in f:\programs\anaconda\lib\site-packages (from python-dateutil>=2.8.2->jupyter-client<9,>=7.4.4->voila) (1.16.0)

Requirement already satisfied: charset-normalizer<4,>=2 in f:\programs\anaconda\lib\site-packages (from requests>=2.28->jupyterlab-server<3,>=2.3.0->voila) (2.0.4)

Requirement already satisfied: urllib3<3,>=1.21.1 in f:\programs\anaconda\lib\site-packages (from requests>=2.28->jupyterlab-server<3,>=2.3.0->voila) (1.26.16)

Requirement already satisfied: certifi>=2017.4.17 in f:\programs\anaconda\lib\site-packages (from requests>=2.28->jupyterlab-server<3,>=2.3.0->voila) (2023.5.7)

Requirement already satisfied: argon2-cffi-bindings in f:\programs\anaconda\lib\site-packages (from argon2-cffi->jupyter-server<3,>=2.0.0->voila) (21.2.0)

Requirement already satisfied: soupsieve>1.2 in f:\programs\anaconda\lib\site-packages (from beautifulsoup4->nbconvert<8,>=6.4.5->voila) (2.4)

Requirement already satisfied: webencodings in f:\programs\anaconda\lib\site-packages (from bleach->nbconvert<8,>=6.4.5->voila) (0.5.1)

Requirement already satisfied: executing in f:\programs\anaconda\lib\site-packages (from stack-data->ipython>=6.1.0->ipywidgets) (0.8.3)

Requirement already satisfied: asttokens in f:\programs\anaconda\lib\site-packages (from stack-data->ipython>=6.1.0->ipywidgets) (2.0.5)

Requirement already satisfied: pure-eval in f:\programs\anaconda\lib\site-packages (f

```

rom stack-data->ipython>=6.1.0->ipywidgets) (0.2.2)
Collecting fqdn (from jsonschema>=4.17.3->jupyterlab-server<3,>=2.3.0->voila)
  Downloading fqdn-1.5.1-py3-none-any.whl (9.1 kB)
Collecting isoduration (from jsonschema>=4.17.3->jupyterlab-server<3,>=2.3.0->voila)
  Downloading isoduration-20.11.0-py3-none-any.whl (11 kB)
Requirement already satisfied: jsonpointer>1.13 in f:\programs\anaconda\lib\site-pack
ages (from jsonschema>=4.17.3->jupyterlab-server<3,>=2.3.0->voila) (2.1)
Collecting uri-template (from jsonschema>=4.17.3->jupyterlab-server<3,>=2.3.0->voila)
  Downloading uri_template-1.3.0-py3-none-any.whl (11 kB)
Collecting webcolors>=1.11 (from jsonschema>=4.17.3->jupyterlab-server<3,>=2.3.0->voi
la)
  Downloading webcolors-24.8.0-py3-none-any.whl (15 kB)
Requirement already satisfied: cffi>=1.0.1 in f:\programs\anaconda\lib\site-packages
(from argon2-cffi-bindings->argon2-cffi->jupyter-server<3,>=2.0.0->voila) (1.15.1)
Requirement already satisfied: pycparser in f:\programs\anaconda\lib\site-packages (f
rom cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi->jupyter-server<3,>=2.0.0->voila)
(2.21)
Requirement already satisfied: arrow>=0.15.0 in f:\programs\anaconda\lib\site-package
s (from isoduration->jsonschema>=4.17.3->jupyterlab-server<3,>=2.3.0->voila) (1.2.3)
Installing collected packages: websockets, webcolors, uri-template, fqdn, isoduratio
n, voila
Successfully installed fqdn-1.5.1 isoduration-20.11.0 uri-template-1.3.0 voila-0.5.7
webcolors-24.8.0 websockets-13.1

```

In []:

```

In [33]: import os
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import transforms
from PIL import Image
import ipywidgets as widgets
from IPython.display import display

# Define CNNModel class
class CNNModel(nn.Module):
    def __init__(self, num_classes=75):
        super(CNNModel, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.fc1 = nn.Linear(64 * 16 * 16, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# Load the trained model
model = CNNModel(num_classes=75)
model.load_state_dict(torch.load('model/my_model.pth', map_location=torch.device('cpu')
model.eval() # Set the model to evaluation mode

```

```

# Define the transformation for input images
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# List of butterfly class indices (this is an example, update with actual class indices)
butterfly_classes = list(range(0, 50)) # Assuming classes 0-49 are butterflies

# Create the widgets
upload_button = widgets.FileUpload(accept='image/*', multiple=False)
predict_button = widgets.Button(description='Predict')
output = widgets.Output()

# Function to preprocess the image and make a prediction
def predict_image(change):
    output.clear_output()
    if upload_button.value:
        # Access the uploaded file from the tuple
        uploaded_file = list(upload_button.value.values())[0] if isinstance(upload_button.value, dict) else upload_button.value
        content = uploaded_file['content']

        # Save the image temporarily
        temp_image_path = 'temp_uploaded_image.png'
        with open(temp_image_path, 'wb') as f:
            f.write(content)

        # Open and preprocess the image
        image = Image.open(temp_image_path).convert('RGB')
        image = transform(image).unsqueeze(0)

        # Make prediction
        with torch.no_grad():
            outputs = model(image)
            _, predicted = torch.max(outputs, 1)

        # Display results
        with output:
            display(Image.open(temp_image_path))
            class_id = predicted.item()
            if class_id in butterfly_classes:
                print(f'Predicted Class: {class_id} - It is a butterfly.')
            else:
                print(f'Predicted Class: {class_id} - It is not a butterfly.')

# Link the predict button to the prediction function
predict_button.on_click(predict_image)

# Display the widgets
app = widgets.VBox([upload_button, predict_button, output])
display(app)

```

C:\Users\Jordan\AppData\Local\Temp\ipykernel_25620\1690436974.py:32: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
model.load_state_dict(torch.load('model/my_model.pth', map_location=torch.device('cpu')))
```

```
VBox(children=(FileUpload(value=(), accept='image/*', description='Upload'), Button(description='Predict', sty...
```

In []:

In []: